Week 1: Security Assessment

- By Hussein Khaled

Objective:

Learn about common web application vulnerabilities by analyzing a simple web application.

Skills:

Basic Web Security, Vulnerability
Identification

Tools:

OWASP ZAP, Webgoat

Web Application Vulnerability:

Web application vulnerabilities are weaknesses in software applications that can be exploited by attackers to compromise the integrity, confidentiality, or availability of the application. Understanding these vulnerabilities is crucial for developers and organizations to protect their systems from potential threats.

Common Web Application Vulnerabilities:

1. SQL Injection:

SQL injection occurs when an attacker inserts malicious SQL commands into a query, allowing them to manipulate database operations.

2. Cross-Site Scripting (XSS):

XSS vulnerabilities allow attackers to inject malicious scripts into web pages viewed by other users. This can lead to session hijacking, defacement of websites, or redirecting users to malicious sites.

3. Cross-Site Request Forgery (CSRF):

CSRF tricks a user into executing unwanted actions on a web application in which they are authenticated.

4. Security Misconfiguration:

This vulnerability arises from improper configuration of security settings in applications and servers.

5. Broken Access Control:

Broken access control occurs when users can act outside their intended permissions, allowing unauthorized access to sensitive data or functionality.

6. Insecure Cryptographic Storage

Sensitive data must be stored securely; however, many applications fail to encrypt sensitive information adequately, making it accessible to attackers if they gain access to the storage.

7. Remote Code Execution (RCE):

RCE vulnerabilities allow attackers to execute arbitrary code on a server by exploiting flaws in the application's code or configuration. This can lead to complete control over the affected server.

8. XML External Entities (XXE)

XXE vulnerabilities occur when XML input containing a reference to an external entity is processed by a weakly configured XML parser. This can lead to data exposure or server-side request forgery.

9. Unrestricted File Upload

When an application allows users to upload files without proper validation, attackers can upload malicious files that may compromise the server or other users.

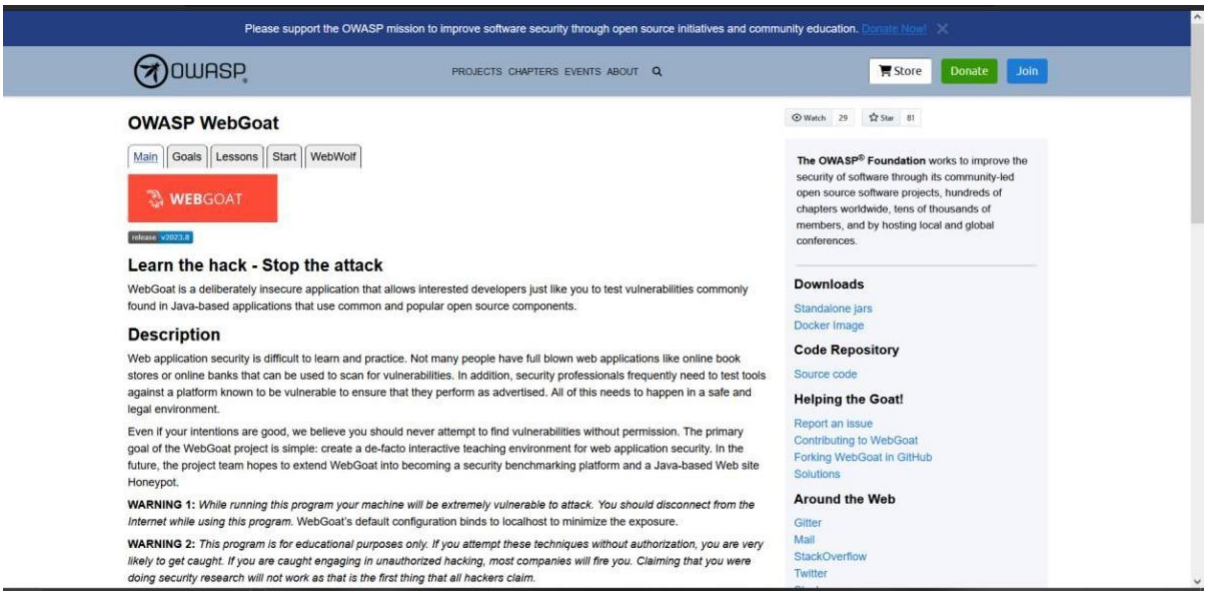
10. Session Management Issues

Weaknesses in session management can lead to session hijacking or fixation attacks, where an attacker takes over a user's session by exploiting poor session ID management practices.

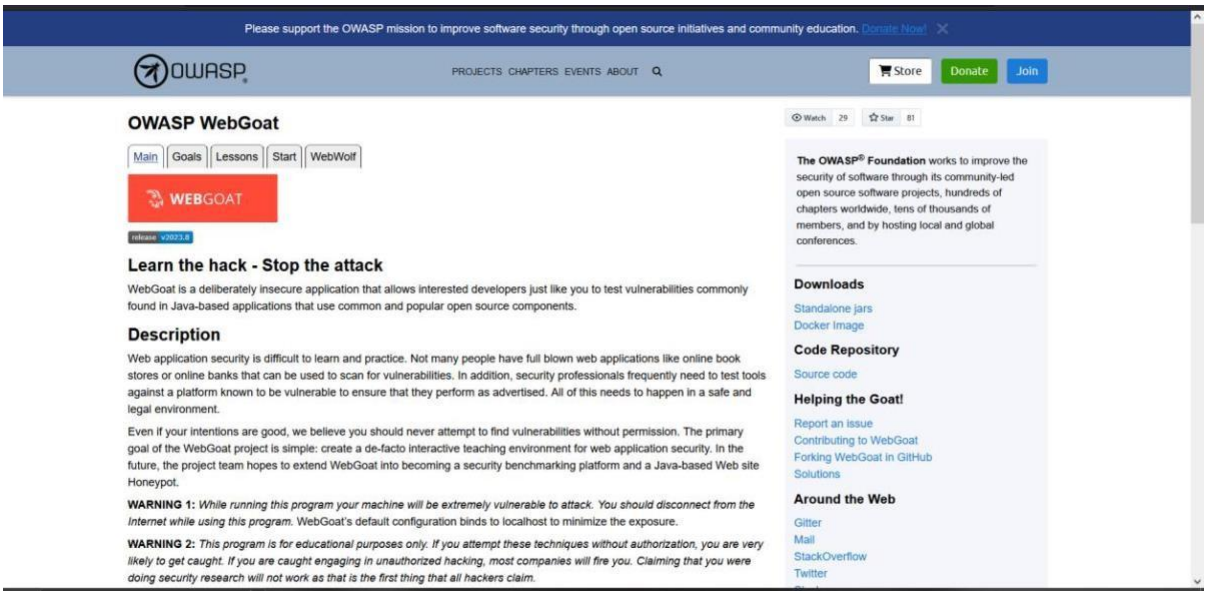
WEBGOAT:

WebGoat is an intentionally insecure web application developed by the Open Web Application Security Project (OWASP) designed to teach web application security concepts through hands-on experience. It provides a safe environment for users to learn about and exploit common vulnerabilities found in Java-based applications.

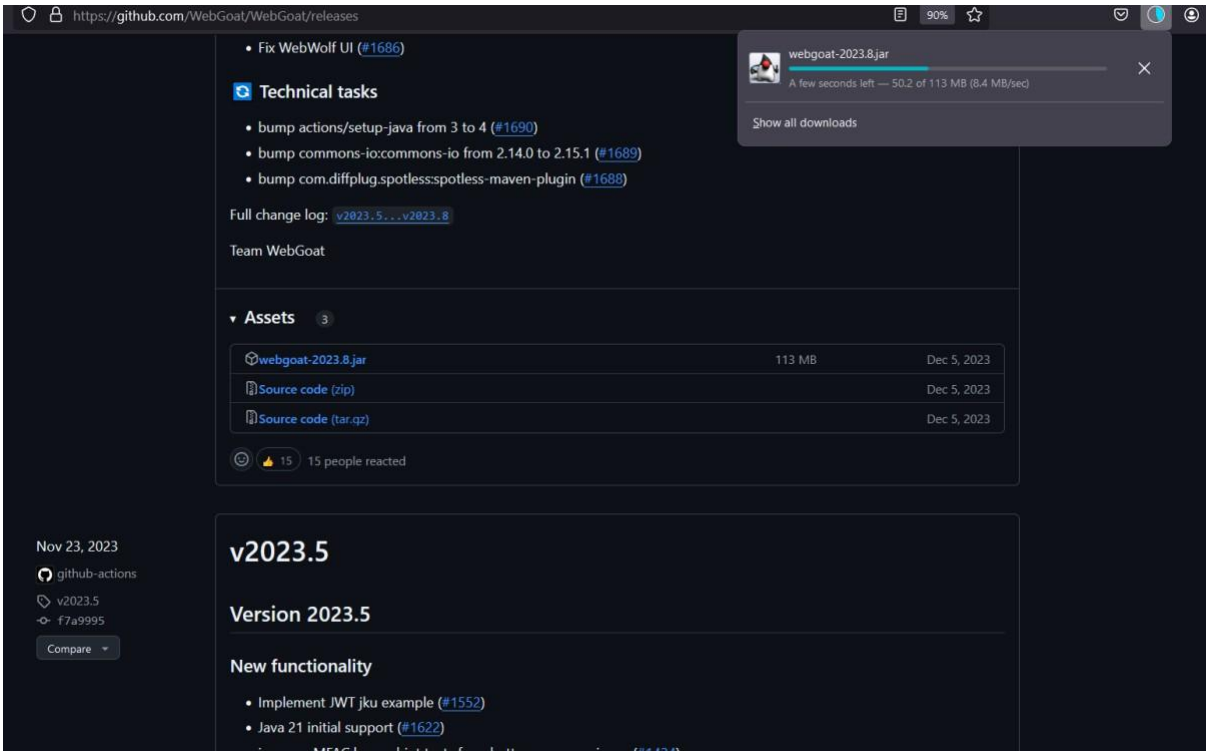
SETTING UP WEBGOAT:



Visit the URL (https://owasp.org/www-project-webgoat/)
Under the Downloads => click "Standalone jars".

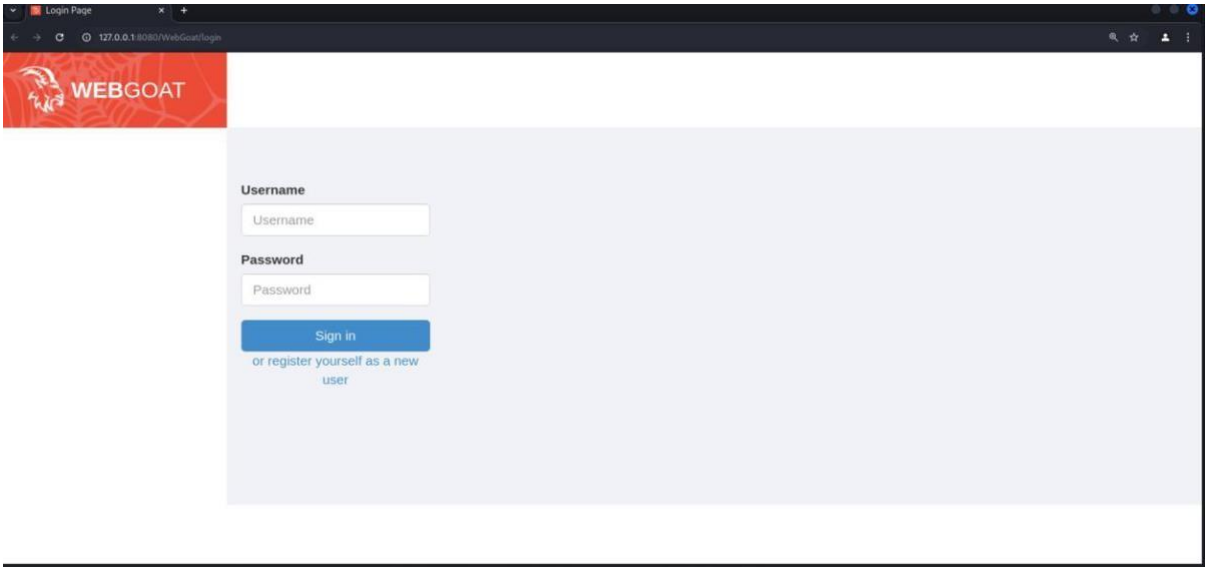


You will be redirected GitHub website and scroll down for this section.



Click the webgoat-2023.8.jar and the download will begin. Run the WebGoat using the java –jar webgoat-<version>.jar

Now open the browser and paste the link give in the screenshot (http://localhost:8080/WebGoat)



SETTING UP OWASP ZAP:

OWASP ZAP (Zed Attack Proxy) is an open-source web application security scanner developed by the Open Web Application Security Project (OWASP).
It is designed to help security professionals and developers identify vulnerabilities in web applications effectively.

FEATURES:

OWASP ZAP functions as a Dynamic Application Security Testing (DAST) tool, meaning it analyzes running applications without needing access to the source code. It can detect a wide range of vulnerabilities, including:

1. SQL Injection

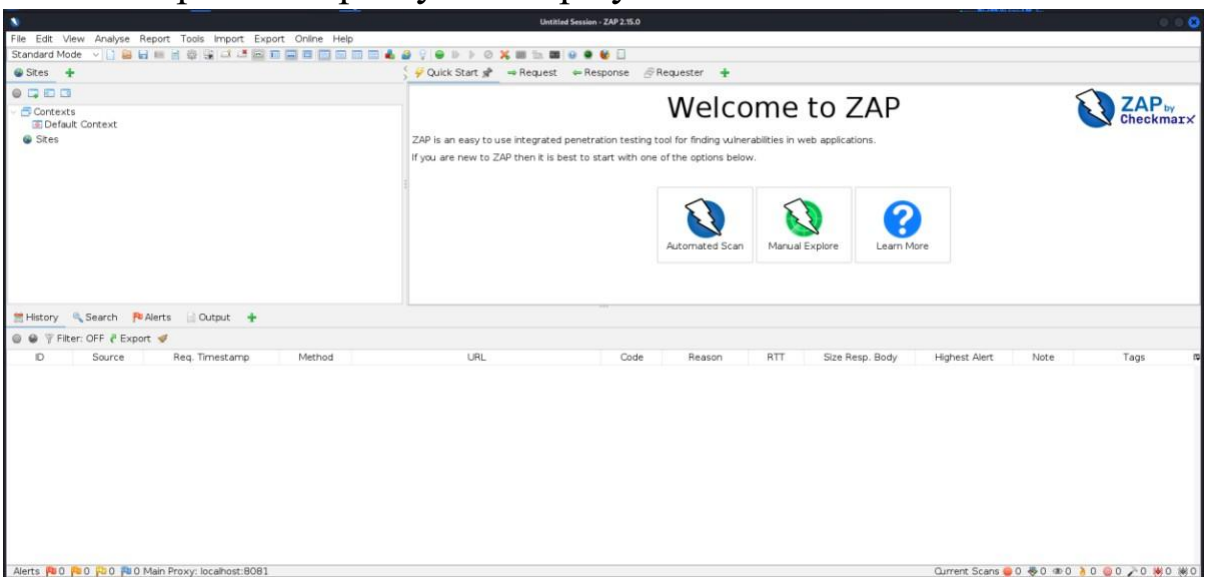
2. Cross-Site Scripting (XSS)
3. Compromised Authentication
4. Exposure of Sensitive Data
5. Security Misconfigurations

SETTING UP ZAPROXY / OWASP ZAP:

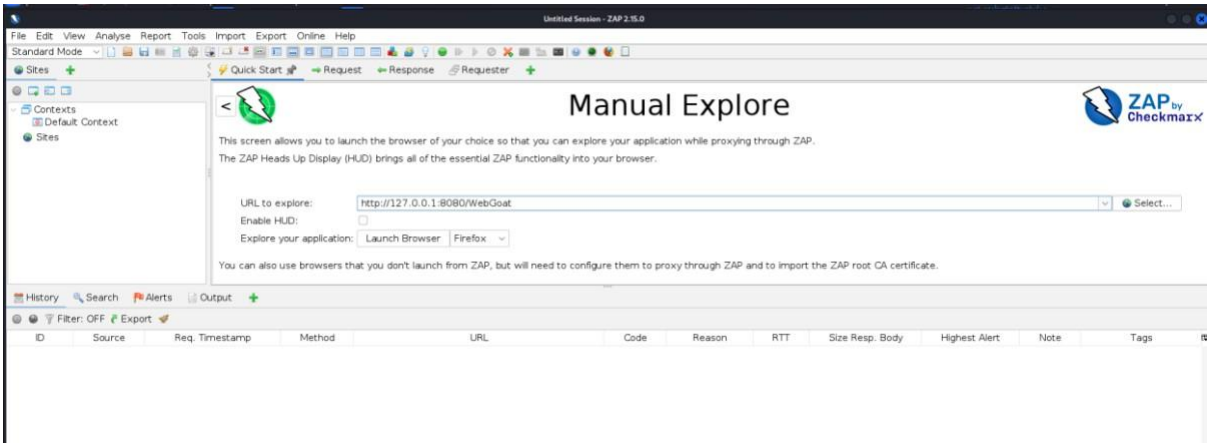
STEP 1: Run "sudo apt install zaproxy"

```
┌──(kali㉿kali)-[~]
└─$ sudo apt install zaproxy
Installing:
  zaproxy

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 307
  Download size: 213 MB
  Space needed: 266 MB / 54.7 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 zaproxy all 2.15.0-0kali1 [213 MB]
Ign:1 http://kali.download/kali kali-rolling/main amd64 zaproxy all 2.15.0-0kali1
Get:1 http://kali.download/kali kali-rolling/main amd64 zaproxy all 2.15.0-0kali1 [213 MB]
5% [1 zaproxy 14.0 MB/213 MB 7%]
```

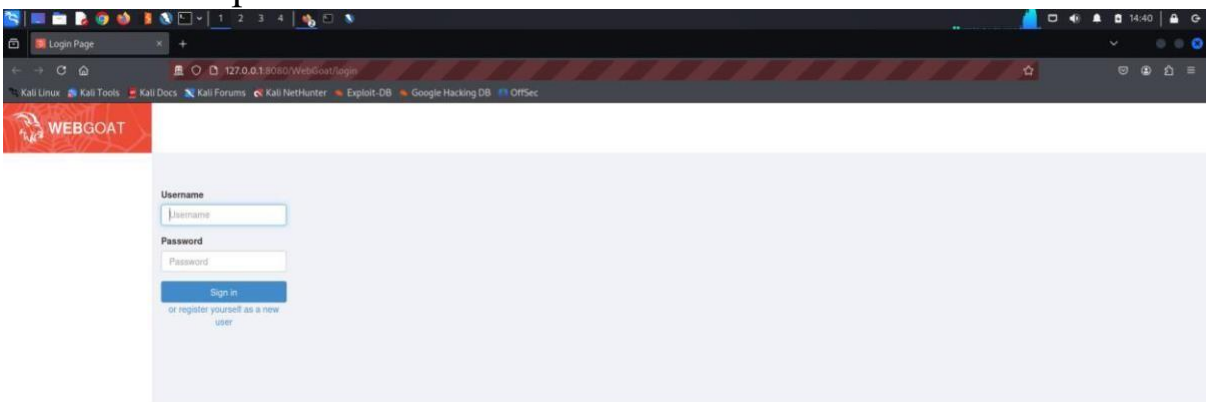
STEP 2: Open the zaproxy and displays like it



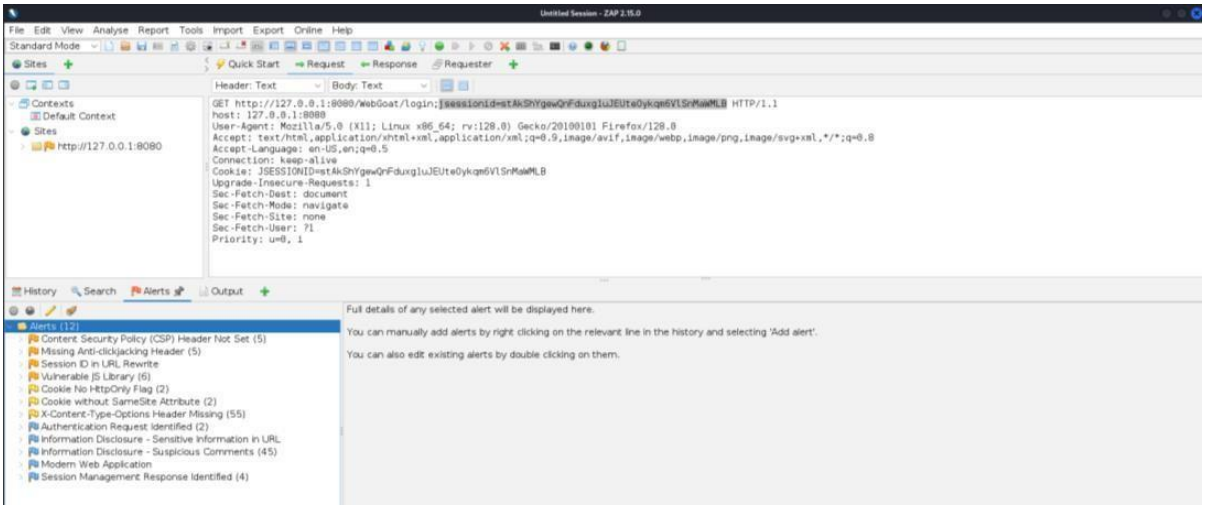
Scanning the localhost website using the Vulnerability scanner (OWASP ZAP):



In Manual Explore launch the browser in Firefox / chrome:

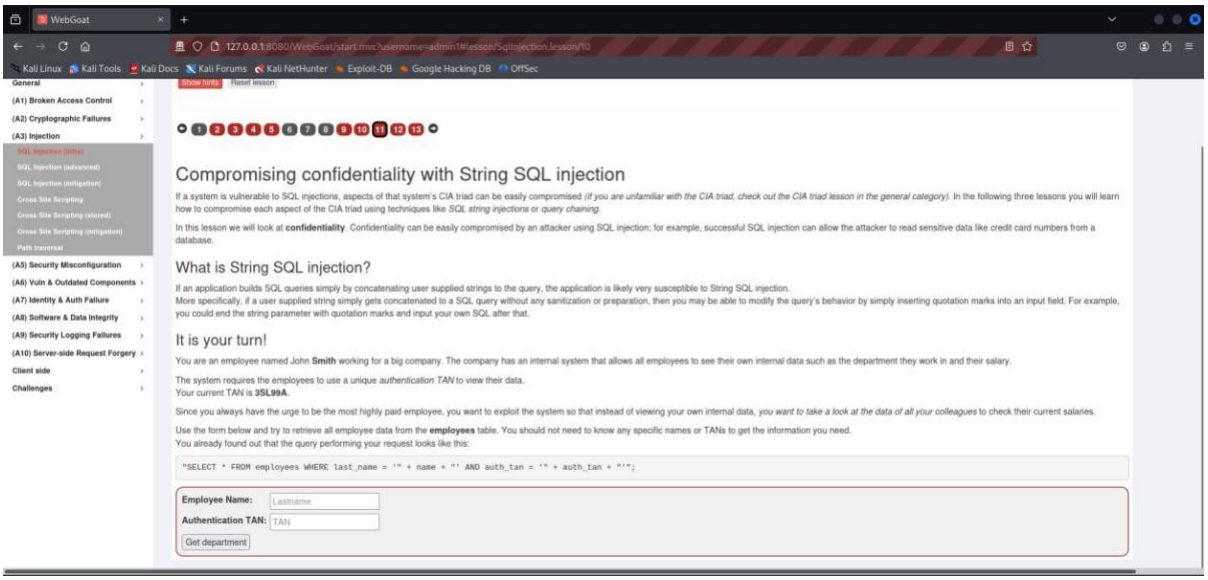


The browser is totally automated by Vulnerability scanner (dark red lines) for capturing the request and response
In "alerts" section we found the vulnerabilities present in the login page.(Demo purpose)

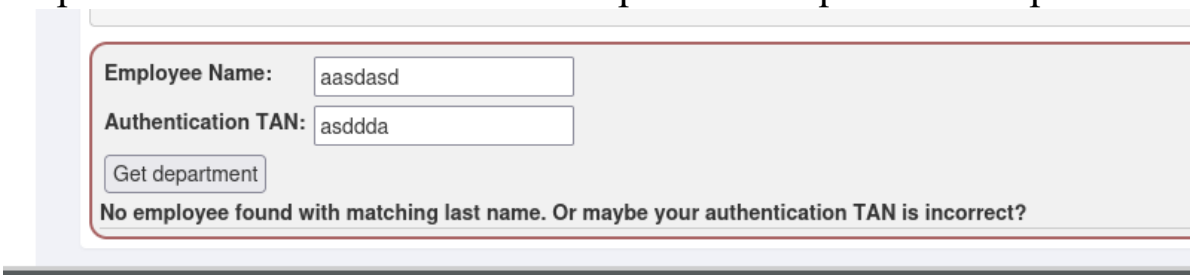


Finding the SQLi vulnerability in WebGoat:

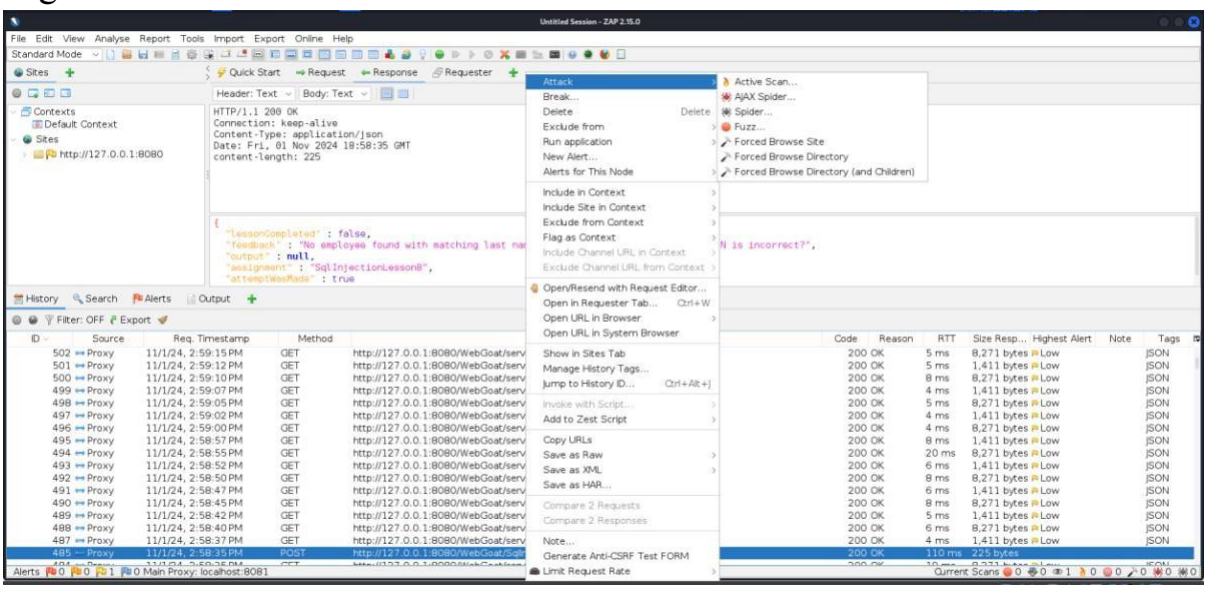
Step 1: Copy the URL and paste the URL in browser.



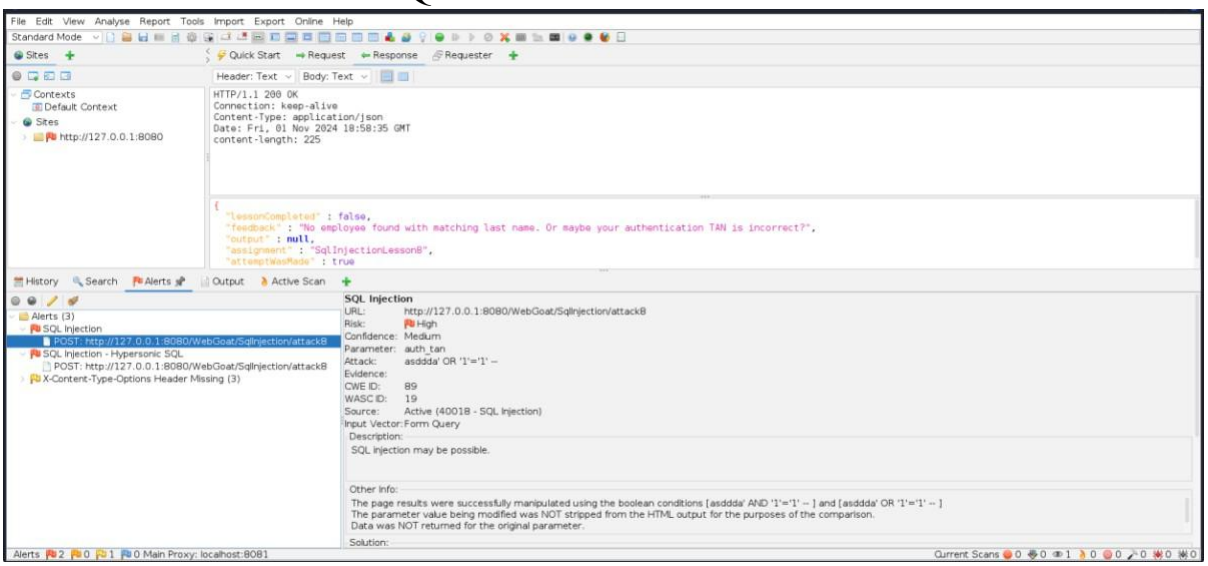
Step 2: Enter the random words to capture the request and response.



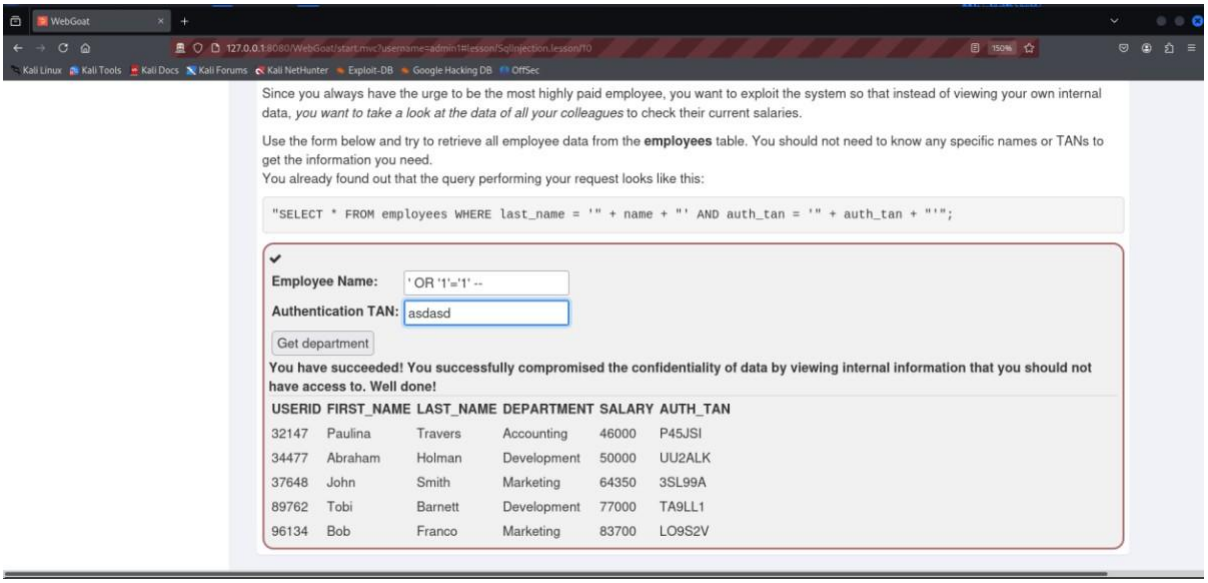
Step 3: Clear the “Alert” section. On the Request of URL
Right click => Attack => Active Scan => Start Scan.



Step 4: After the 100% scan navigate to the “Alert” section for result of alerts. We found the “SQLi” vulnerabilities.



Step 5: Implement in the WebGoat.



Consequences of SQL Injection:

Unauthorized Data Access:

Attackers can bypass authentication mechanisms, gaining unauthorized access to sensitive data, including personal information, financial records, and trade secrets. This can lead to significant data breaches and identity theft.

Data Manipulation:

SQLi allows attackers to modify or delete records in a database. For instance, they can alter account balances in financial applications or delete critical data, compromising data integrity and operational functionality.

Complete Database Control:

A successful SQL injection can grant attackers full control over the database server. This includes executing administrative operations such as creating new user accounts with elevated privileges or even dropping entire tables2

Escalation of Attacks:

If the database server is compromised, attackers may leverage this access to infiltrate other systems behind firewalls.

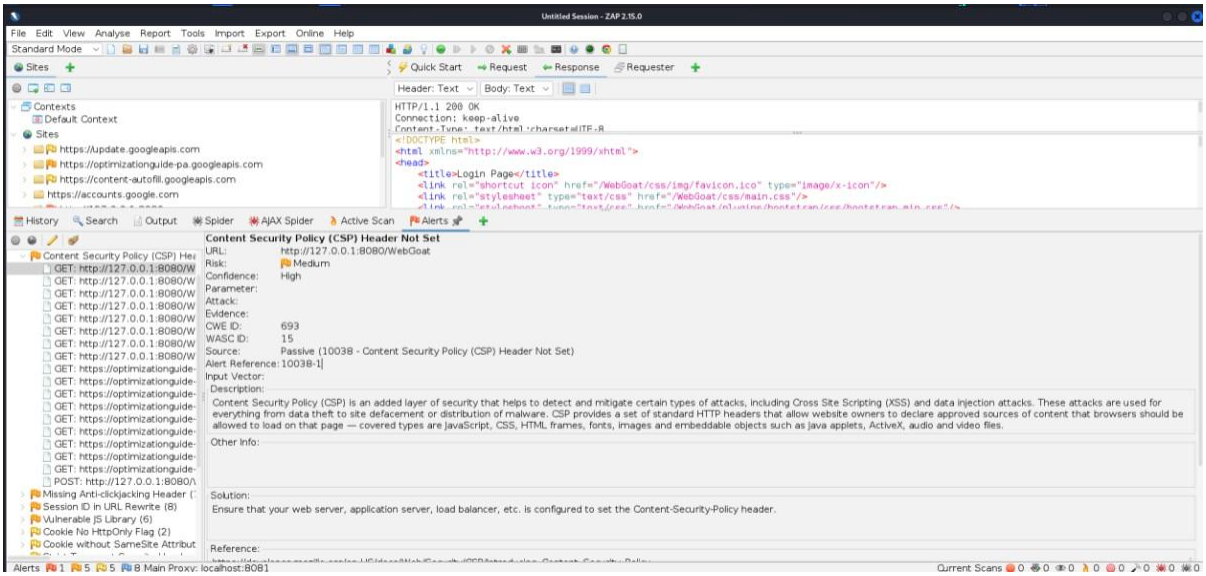
Reputation Damage:

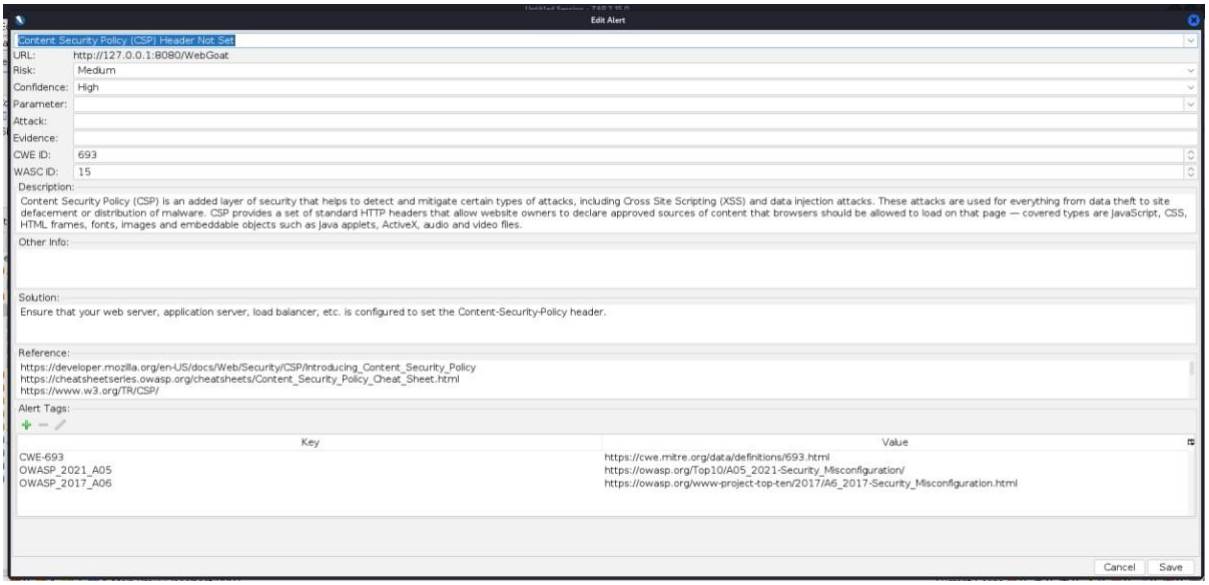
The fallout from an SQL injection attack can severely damage a business's reputation. Customers may lose trust in the organization, leading to loss of business and long-term financial repercussions.

Prevention from the SQLi:

1. Implement Input Validation and Sanitization
2. Use Prepared Statements (Parameterized Queries)
3. Employ Stored Procedures
4. Restrict Database User Privileges
5. Use Web Application Firewalls (WAF)
6. Conduct Regular Security Audits
7. Keep Software Updated

XSS: (Cross Site Scripting)





Open the WebGoat and copy the URL of the WebGoat login page and paste in the Automated Scan URL header option and start the scan.

Content Security Policy (CSP) Header Not Set: Content Security Policy (CSP) is missing, which limits sources from which content (scripts, images, etc.) can be loaded. Without CSP, the application is more vulnerable to attacks like Cross-Site Scripting (XSS), where attackers can inject malicious scripts that get executed in the user's browser.

Consequence of XSS:

Session Hijacking

Attackers steal session cookies to impersonate users, gaining unauthorized access to accounts.

Credential Theft

XSS can be used to trick users into entering login details on fake pages.

Data Theft
Sensitive information on the page can be accessed and stolen by attackers.

Malware Distribution
Malicious code injected through XSS can redirect users to phishing sites or download malware.

Phishing and Social Engineering
XSS can display fake interfaces, tricking users into revealing personal information.

Defacement of Websites
Attackers can alter the appearance of web pages, damaging credibility and reputation.

Denial of Service
XSS may overload the server, making the site inaccessible to legitimate users.

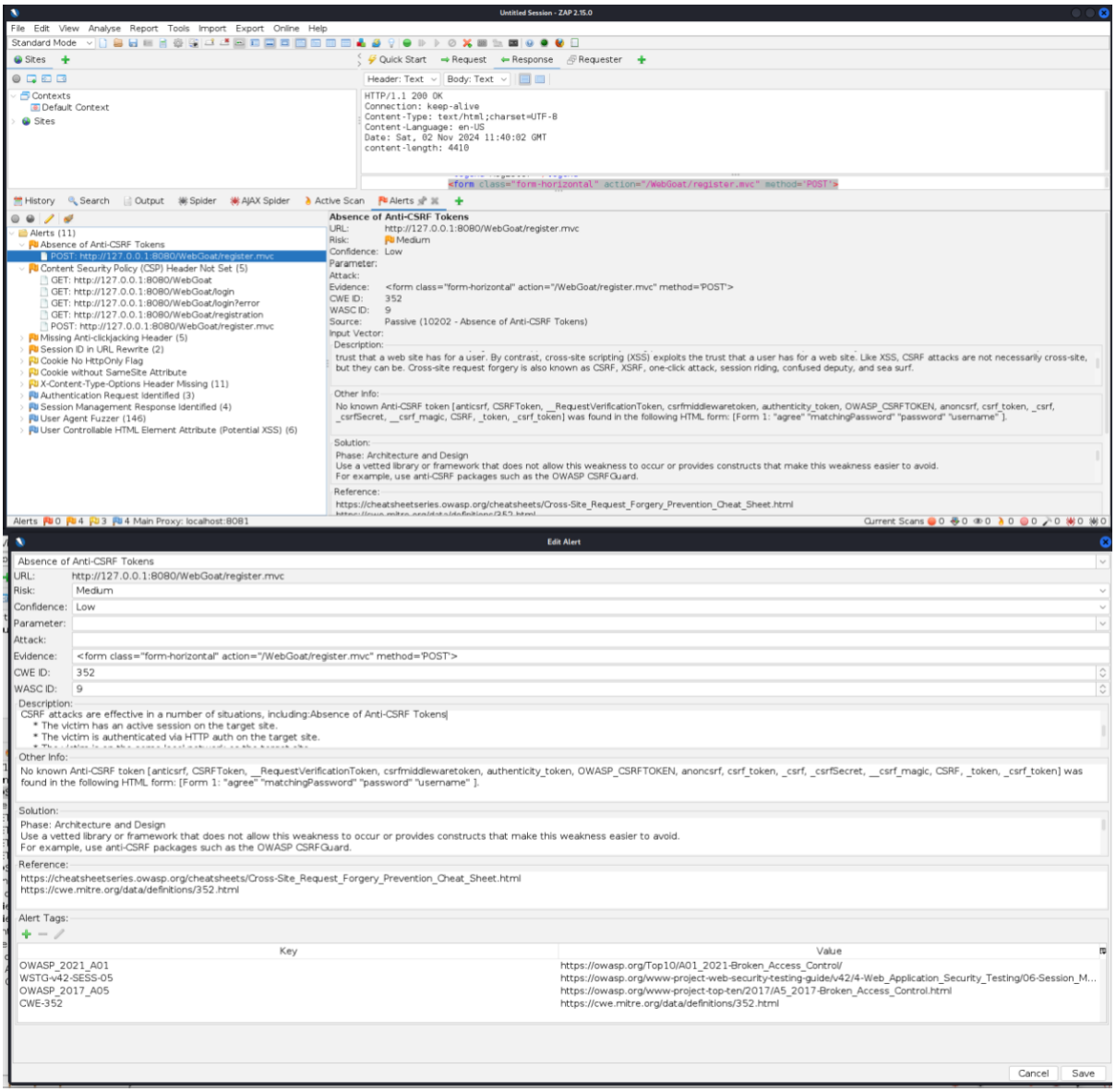
Impacts on SEO and Reputation
Hidden spam links or malware can harm a site's SEO and get it blacklisted.

Prevention:

1. Sanitize and validate user input on both the client and server sides.
2. Use Content Security Policy (CSP) to limit where scripts can be loaded from.
3. Encode output (especially when rendering user-generated content in HTML).
4. Use secure headers such as X-Content-Type-Options, X- Frame-Options, and X-XSS-Protection.

CSRF: (Cross Site Request Forgery)

Open the WebGoat and copy the URL of the WebGoat login page and paste in the Automated Scan URL header option and start the scan



Absence of Anti-CSRF Tokens

The application does not use Anti-CSRF tokens, which protect against Cross-Site Request Forgery (CSRF) attacks. A hacker could trick an authenticated user into performing unintended actions on the application (like transferring funds or changing settings) by exploiting the user's authenticated session.

Consequence of CSRF:

Unauthorized Actions on Behalf of Users:
Attackers can modify settings, change passwords, or initiate transactions without user consent.

Privilege Escalation and Account Takeover:
Attackers may gain unauthorized access or escalate privileges to control accounts.

Data Leakage or Manipulation:
Sensitive data can be exposed or modified, leading to data integrity issues.

Financial Loss:
Unauthorized transactions can cause direct financial harm to victims.

Reputational Damage:
Breaches due to CSRF attacks can damage the organization's reputation.

Loss of User Trust:
Users may lose confidence in the platform after unauthorized actions occur.

Legal and Compliance Issues:
CSRF vulnerabilities can result in regulatory violations and fines.

Prevention:
Anti-CSRF Tokens:
Generate unique tokens for each session and validate them on the server to confirm request authenticity.

SameSite Cookies:
Set cookies with the SameSite attribute to restrict cross-site requests.

User Authentication Validation:
Ensure that critical actions require reauthentication or confirmation from the user.

Implement Strong CORS Policies:
Restrict cross-origin requests to trusted sources.

Use Secure Headers:
Enforce X-Frame-Options and Content-Security-Policy headers to limit content injection and clickjacking.

Educate Users:
Inform users to avoid suspicious links and to log out when not actively using the application.

Conclusion:

In conclusion, this internship has been a rewarding experience, allowing me to deepen my understanding of network security, firewall configuration, and cybersecurity fundamentals. I've had the chance to apply theoretical knowledge to practical tasks, enhancing my skills in threat identification, network monitoring, and secure configurations. This hands-on experience has strengthened my problem-solving abilities and broadened my perspective on real-world security challenges. I'm grateful for the opportunity to grow and contribute, and I look forward to applying what I've learned to make a positive impact in the cybersecurity field. Thank you for this valuable learning experienc